

GUÍA VISUAL DE ESTUDIO

Evaluación Sumativa N° 2 (35%) · Programación Back End

INACAP Sede Maipú · Primavera 2026



Estudiante: Alexis Ferrada



Docente: Javier Eduardo Gutiérrez Osorio



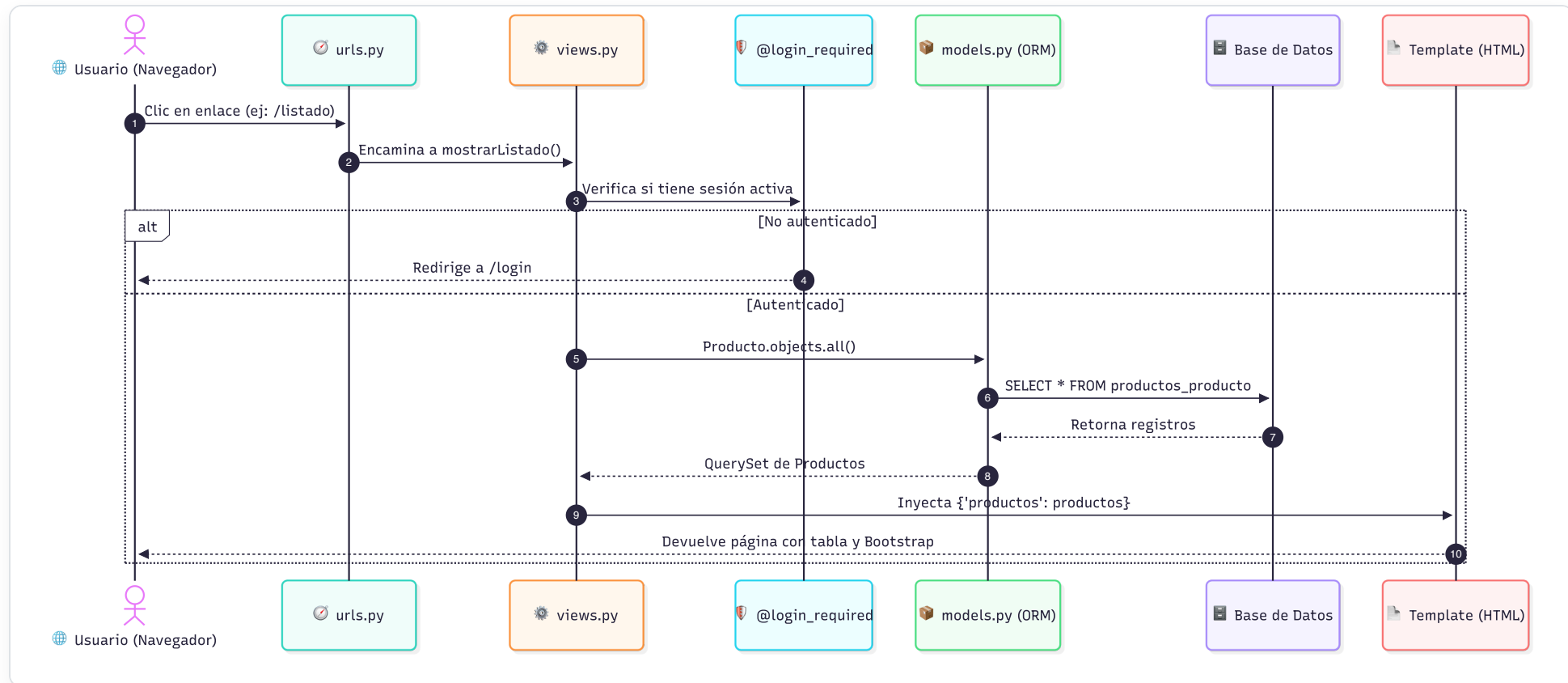
Puntaje Interrogación: 30 Puntos (Explicación del Código)



Proyecto: `Proyecto_BD` (Django 5.2 + MySQL / SQLite + Docker)

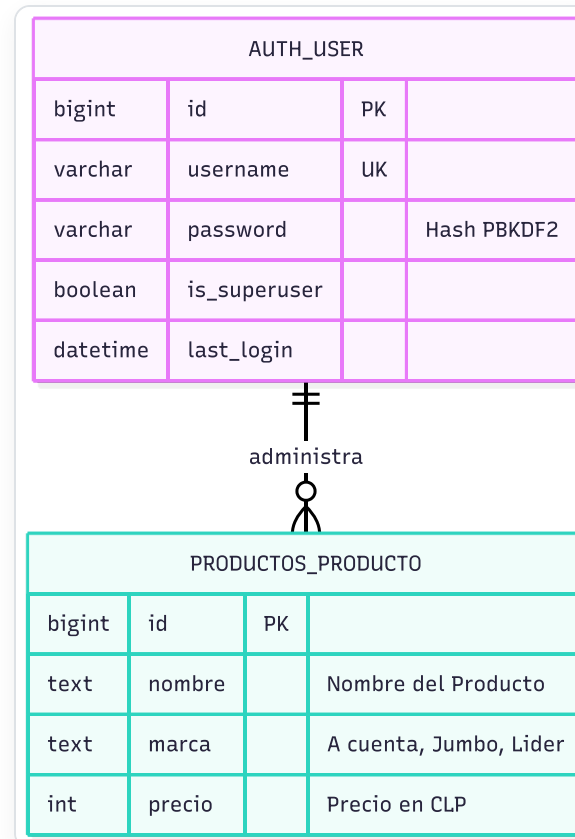
1. Ciclo de Vida de una Petición (Arquitectura MVT)

¿Cómo viaja la información? El usuario solicita una URL → `urls.py` deriva a `views.py` → `@login_required` valida sesión → El ORM consulta a la Base de Datos → La vista inyecta los datos en el Template HTML → Se muestra en pantalla.



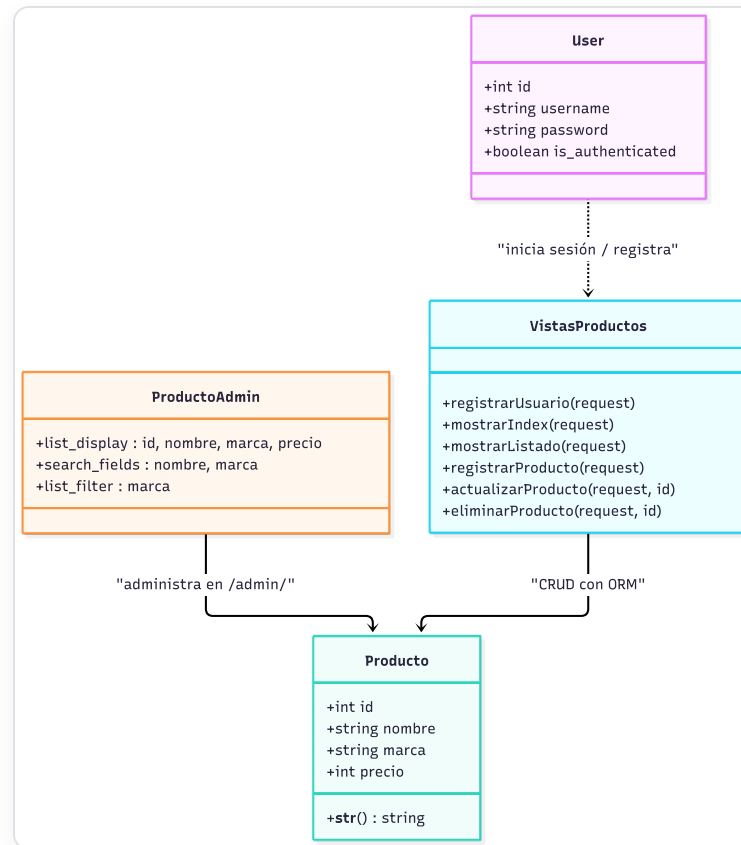
2. Estructura de la Base de Datos (Diagrama Entidad-Relación)

Dos tablas principales: `auth_user` (creada automáticamente para autenticación y login) y `productos_producto` (creada con el modelo Producto para el CRUD).



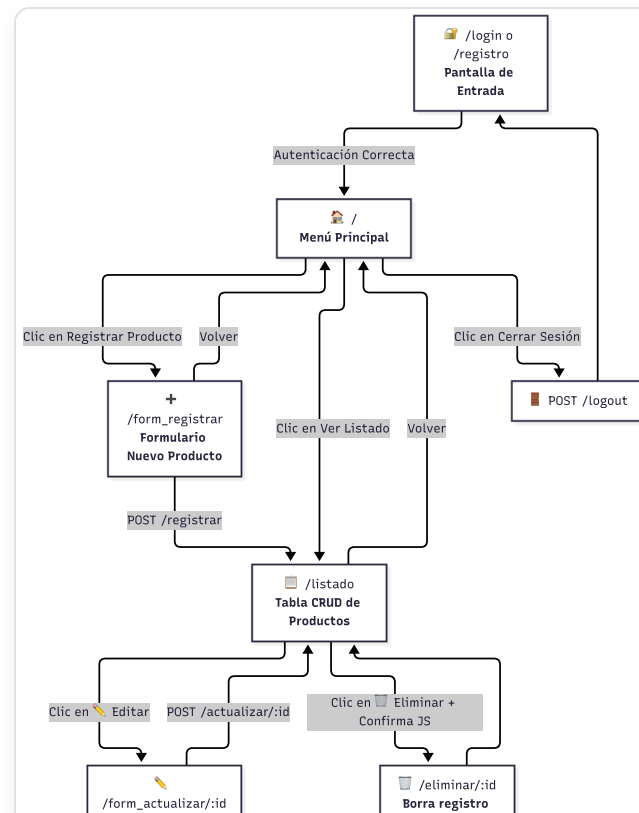
3. Diagrama de Clases y Controladores (UML)

Estructura de Objetos: `Producto` define los datos y se registra en `ProductoAdmin`. Las vistas funcionales controlan las operaciones CRUD llamando al ORM de Django.



4. Mapa Mental de Navegación (Sitemap de Pantallas)

Flujo de usuario: La pantalla principal unifica Login y Registro → Al ingresar entra al Menú Principal → Desde allí se accede a Registrar Producto o Listado → Editar y Eliminar retornan siempre al Listado.



5. Arquitectura de Despliegue: Docker vs Modo Local



Modalidad Docker Compose

- **Contenedor Web (:8000):** Django 5.2 sobre Python 3.12.
- **Contenedor BD (:3307):** Servidor MySQL 8.0 con healthcheck.
- **Contenedor phpMyAdmin (:8080):** Interfaz visual de gestión.
- **Volumen persistente:** `mysql_data` guarda los datos aunque se apague el contenedor.
- **Red interna:** Los servicios se comunican usando el nombre de host `db:3306`.



Modalidad Local (Sin Docker)

- **Motor:** SQLite3 (archivo `db.sqlite3`).

6. Tarjeta Rápida para la Interrogación (30 Puntos)

Pregunta del Profesor	Respuesta Exacta y Concisa
¿Qué es el ORM?	El mapeador de objetos de Django. Traduce clases y métodos Python (<code>.all()</code> , <code>.create()</code> , <code>.delete()</code>) a SQL sin escribir consultas a mano y previene inyecciones SQL.
¿Cómo se protegen las rutas?	Con el decorador <code>@login_required</code> arriba de cada vista. Si el usuario no tiene sesión activa, lo envía a <code>/login</code> .
¿Para qué sirve <code>{% csrf_token %}</code>	Protege los formularios POST contra falsificación de petición en sitios cruzados mediante un token